arXiv:2502.13063v3 [cs.CL] 22 Jun 2025

Cramming 1568 Tokens into a Single Vector and Back Again: Exploring the Limits of Embedding Space Capacity

Yuri Kuratov[1,2] **Mikhail Arkhipov**[3] **Aydar Bulatov**[1,2] **Mikhail Burtsev**[4]

[1]AIRI, Moscow, Russia
[2]Neural Networks and Deep Learning Lab, MIPT, Dolgoprudny, Russia
[3]Independent Researcher, Amsterdam, Netherlands
[4]London Institute for Mathematical Sciences, London, UK

Correspondence: yurii.kuratov@phystech.edu

Abstract

A range of recent works addresses the problem of compression of sequence of tokens into a shorter sequence of real-valued vectors to be used as inputs instead of token embeddings or key-value cache. These approaches are focused on reduction of the amount of compute in existing language models rather than minimization of number of bits needed to store text. Despite relying on powerful models as encoders, the maximum attainable lossless compression ratio is typically not higher than x10. This fact is highly intriguing because, in theory, the maximum information capacity of large real-valued vectors is far beyond the presented rates even for 16-bit precision and a modest vector size. In this work, we explore the limits of compression by replacing the encoder with a per-sample optimization procedure. We show that vectors with compression ratios up to x1500 exist, which highlights two orders of magnitude gap between existing and practically attainable solutions. Furthermore, we empirically show that the compression limits are determined not by the length of the input but by the amount of uncertainty to be reduced, namely, the cross-entropy loss on this sequence without any conditioning. The obtained limits highlight the substantial gap between the theoretical capacity of input embeddings and their practical utilization, suggesting significant room for optimization in model design.

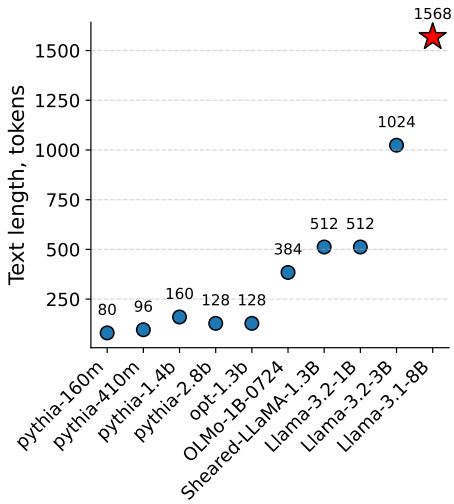

Figure 1: **How many tokens fit into a single input vector?** We estimate maximum number of tokens that can be decoded from a single input vector across various language models.

1 Introduction

Most large language models (LLMs) are built on the Transformer architecture (Vaswani et al., 2017) and have demonstrated remarkable performance as their parameters scale (Radford et al., 2019; Brown et al., 2020; Kaplan et al., 2020; Hoffmann et al., 2022). As model sizes increase, so does the dimensionality of their input embeddings. However, despite this growth, each embedding still represents only a single token, e.g., for a series of Llama models embeddings size is growing from 2,048 in 1B-parameter models to 16,384 float numbers in 405B-parameter models (Dubey et al., 2024). Remarkably, even a 2,048-dimensional vector of 16-bit floats has a theoretical capacity of 32,768 bits, which is sufficient to encode roughly 1,931 tokens from a vocabulary of size 128,256. *This observation motivates us to explore whether language models can utilize the latent capacity of input vectors more effectively, potentially encoding and processing multiple tokens with a single vector.*

Encoding multiple tokens or even entire texts into a compact latent representation has been a longstanding challenge in natural language processing. It includes approaches, such as sentence embeddings (Le and Mikolov, 2014; Kiros et al., 2015; Cer et al., 2018; Wang et al., 2024) for semantic search and retrieval, and text autoencoders (Bowman et al., 2016; Miao et al., 2016; Montero et al., 2021), aimed to capture the essential meaning of texts in a dense representations.

In the context of LLMs, the challenge of encoding prompts and long contexts is particularly important because of the quadratic computational cost of the self-attention mechanism in Transformers. Several works have explored the possibility of replacing token-based prompts with a smaller set of dense vectors (Lester et al., 2021; Li and Liang, 2021; Gao, 2024; Li et al., 2024b), thereby shortening the input sequence and reducing the computational budget. These methods have demonstrated token-to-vector lossy compression ratios on the order of x500 with 8B-parameter models, indicating that it is possible to retain the critical information in a significantly reduced number of vectors. However, lossless compression is still limited by approximately factor of 10.

In memory-augmented architectures (Weston et al., 2015; Sukhbaatar et al., 2015; Burtsev et al., 2021), these embeddings act as additional storage or as a recurrent state for passing information between time steps (Dai et al., 2019; Bulatov et al., 2022; Chevalier et al., 2023; Behrouz et al., 2024), essentially serving as an episodic memory. Moreover, recent approaches have explored the power of latent space reasoning (Hao et al., 2024) where high-capacity embeddings enable models to perform complex multi-step tasks directly in latent space. Consequently, the capacity of these vectors is crucial not only for efficient input representation, but also for increasing the overall expressiveness and computational power of models (Merrill and Sabharwal, 2023; Strobl et al., 2024; Sanford et al., 2024). Better understanding of the latent capacity of input vectors, could significantly help to improve encoding and retrieving of contextual information, episodic memory, as well as complex reasoning within large language models.

In this work, we investigate the limits of such input representations, exploring their capacity to encode and reconstruct long text sequences. By systematically quantifying how much additional information these vectors can capture, we provide insights into the efficiency and potential of latent representations in LLMs. Our main contributions:

1. We empirically study capacity limits of LLM's input representations by compressing texts into trainable `[mem]` vectors.

2. We establish a direct connection between the latent capacity of input vectors and text cross-entropy, providing a quantitative measure of the information each vector can encode.

3. We show that the capacity limits remain consistent across different text lengths and domains, including natural text and random word sequences.

4. We introduce a set of metrics that decouple the capacity of trainable input vectors from the language model's inherent prediction abilities. Using these metrics, we demonstrate a nearly linear scaling of compression capacity with the number of trainable vectors (e.g., Llama-3.2-1B compresses 7,168 tokens into just 16 vectors).

Our code is available at this URL.

2 Related Work

Approaches to compressing LLM context into a shorter sequence of input or KV-cache vectors are explored for various purposes, yet no standardized terminology or unified methodology has emerged.

Context compression. One application for input compression is connected with efficient processing of long contexts with LLMs. RMT (Bulatov et al., 2022) and AutoCompressors (Chevalier et al., 2023) train the whole language model in a recurrent manner to compress the information from input segments to summary vectors and later reuse them to solve long-context tasks. ICAE Ge et al. (2024) uses an autoencoder architecture with a frozen LLM as a decoder and adapt the same LLM for the encoder using LoRA (Hu et al., 2022). The resulting pipeline is pretrained using autoencoding and language modeling objectives, and then finetuned for language tasks, achieving the effective compression rate of x4. SelfCP (Gao, 2024) uses the base LLM itself as a compressor using a trainable adapter to aggregate compressed states across multiple segments. 500xCompressor (Li et al., 2024b) extends the autoencoding approach with layer-wise connections and additional language pretraining tasks, exploring compression ratios up to x480, though at the cost of substantial quality degradation. Alternative approaches aim to compress KV activations instead of input tokens. Some methods achieve this by estimating token relevance, either through training-free (Zhang et al., 2023; Li et al., 2024a) or training-based approaches (Qin and Van Durme, 2023; Qin et al., 2024), to prune irrelevant tokens and focus computation on the most informative ones. These strategies can yield high-quality but lossy compression with ratios up to x20. This result can be improved by finetuning models to leverage the resulting cache representations more effectively. This way, KV-Distill (Chari et al., 2025) can reduce cache size up to 100 times with

negligible loss in QA performance. In contrast, our method, applied to models of comparable size (up to 8 billion parameters), demonstrates that a compression rate of x1568 can be achieved without any loss in reconstruction quality.

Prompt compression. Another line of work targets prompts compression to reduce inference costs. Gist tokens (Mu et al., 2023) are prompt representations compressed by the LLM itself, finetuned with a special mask. Gisting allows to achieve prompt compression rate up to x26 with only minor loss in model performance. LLMLingua (Jiang et al., 2023) decouples the compression operation from the LLM and introduces a coarse-to-fine prompt compression strategy with a budget controller and token-level iterative compression, achieving up to x20 compression with negligible performance loss. Jiang et al. (2024); Pan et al. (2024) extend this framework to long contexts, improving information retention through data distillation. Additionally, Morris et al. (2023, 2024) show that the original text can be reconstructed not only from its embeddings but even from the LM's predictions. In the current work we apply a per-sample optimization process instead to explore the fundamental limits of compression and establish upper bounds on compression rates that far exceed prior work.

LLM-based lossless compression pipelines. Language models have also been investigated for lossless text compression.[1] LLMZip (Valmeekam et al., 2023) improves standard compression by ranking candidates using next-token probabilities, while FineZip (Mittu et al., 2024) accelerates compression through fine-tuning and dynamic context management for better efficiency. The capabilities of LLMs in compression pipelines can be measured in bits-per-token over a representative textual corpus. Huang et al. (2024); Guo et al. (2024) provide such measurements for public LLMs and establish the connection between compression rate and model performance, measured by diverse benchmark scores. Unlike these methods, we do not rely on external compression algorithms. Instead, we achieve lossless compression using only the LLM itself, providing both theoretical insights and practical demonstrations of compression limits.

[1] Delétang et al. (2024) explore data compression (texts to sequences of bits) using Arithmetic Coding and LLMs. This line of research is orthogonal to ours, as the sequence of bits cannot be passed as an input to an LLM. We analyze our approach from the data compression perspective in Appendix F.

Trainable tokens. Some works use the trainable input tokens in other ways. Burtsev et al. (2021) uses memory tokens as additional representation storage, Beltagy et al. (2020); Zaheer et al. (2020) use similar global tokens to enhance long-range information flow. Li and Liang (2021); Lester et al. (2021); Gao (2024); Liu et al. (2022) explore trainable soft prompts as an alternative to finetuning model weights. Our findings about the representation capacity can represent the potential efficiency limits of such methods, based on how far the model behavior can be changed using trainable tokens.

3 Method

We propose a simple approach for compressing a sequence of tokens into a small set of "memory" vectors. Then with this method we analyze how many tokens can be stored and decoded from a small set of resulting vectors. Fig. 2 provides an overview of our setup.

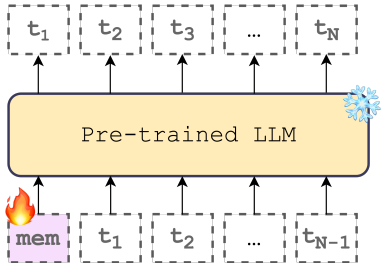

Figure 2: **Compressing text into a [mem] vector.** The pre-trained LLM is frozen, and we only finetune one or multiple [mem] vectors to decode the sequence of tokens $[t_1, t_2, \ldots, t_N]$. [mem] vectors are trained for each text separately.

Trainable [mem] vectors are inspired by Memory Transformers (Burtsev et al., 2021), but here these vectors are designed to encode an entire text sequence. The training method is similar to Prompt Tuning (Lester et al., 2021), with only a set of special input embeddings optimized while all parameters of the language model are frozen.

Formally, given a token sequence $[t_1, t_2, \ldots, t_N]$, we introduce a set of trainable vectors [mem] $= [m_1, \ldots, m_K]$ that are prepended to the text. These [mem] vectors are optimized to encode $[t_1, t_2, \ldots, t_N]$. During training, the frozen language model processes $[m_1, \ldots, m_K, t_1, t_2, \ldots, t_i]$ as the input context for predicting next token t_{i+1}. The [mem] vectors are optimized by minimizing the standard next-token prediction cross-entropy loss. As a result, each text sequence is associated with a unique